

Proyecto #1 – Haskell

Almacén Robótico

En un almacén robótico altamente automatizado, ha ocurrido un error en el sistema de clasificación. Una "Caja Objetivo" de vital importancia ha quedado atrapada entre otras cajas ordinarias.

Su objetivo como especialista en programación funcional es diseñar un programa que controle a un Robot de carga para empujar la **Caja Objetivo** hasta la zona de extracción, encontrando la secuencia mínima de movimientos necesarios. El programa debe considerar las siguientes reglas y restricciones del entorno:

1. El almacén se representa como una matriz fija de 6x6 (coordenadas de la (0,0) a la (5,5)). La casilla (0,0) es la esquina superior izquierda.
2. Existen tres tipos de entidades en el tablero: Un **Robot**, una **Caja Objetivo**, y múltiples **Cajas de Bloqueo** (obstáculos). Cada entidad ocupa exactamente una casilla.
3. El Robot puede moverse en cuatro direcciones: Arriba (U), Abajo (D), Izquierda (L) y Derecha (R).
4. Si el Robot se mueve hacia una casilla vacía, simplemente cambia de posición.
5. **Mecánica de Empuje:** Si el Robot se mueve hacia una casilla ocupada por una caja (sea la Objetivo o de Bloqueo), el Robot empujará la caja una casilla en la misma dirección, **si y solo si** la casilla detrás de la caja está vacía y dentro de los límites del tablero.
6. **Prohibición de empuje múltiple:** El Robot no tiene fuerza para empujar dos cajas al mismo tiempo. Si detrás de la caja que se desea empujar hay otra caja, el movimiento es inválido.
7. Ninguna entidad puede salirse de los límites del tablero.
8. La zona de extracción (la meta) es siempre la coordenada **(5,5)**. El juego termina exitosamente en el instante en que la **Caja Objetivo** llega a esa coordenada.
9. Se debe minimizar la cantidad de movimientos del Robot (mejor solución).

Parte 1 – Inicialización del Estado

El estado del juego almacena la posición del Robot, la posición de la Caja Objetivo y una lista con las posiciones de las Cajas de Bloqueo.

type Coord = (Int, Int) -- (Fila, Columna)

data Move = U | D | L | R deriving (Show, Eq)

type State = (Coord, Coord, [Coord]) -- (Robot, CajaObjetivo, CajasDeBloqueo)

Escriba la función `initialState` que reciba la coordenada inicial del Robot, la de la Caja Objetivo, y la lista de Cajas de Bloqueo, devolviendo el estado `State`. Se debe validar que **ninguna coordenada se salga del tablero (0 a 5)** y que **ninguna entidad se solape con otra**. Si la configuración es inválida, devuelva un estado de error estandarizado: `((-1,-1), (-1,-1), [])`.

initialState :: Coord -> Coord -> [Coord] -> State

Caso de prueba:

1) Caso Normal: Todo dentro de los límites y sin solaparse.

initialState (0,0) (2,2) [(1,1), (3,3)]

Retorno: ((0,0), (2,2), [(1,1), (3,3)])

2) Caso Solapamiento: El Robot y la Caja Objetivo inician en la misma casilla.

initialState (1,1) (1,1) [(3,3)]

Retorno: ((-1,-1), (-1,-1), [])

3) Caso Fuera de Límites: La coordenada del Robot excede la matriz 6x6.

initialState (6,0) (2,2) []

Retorno: ((-1,-1), (-1,-1), [])

Parte 2 - Validación de Movimientos

Antes de ejecutar un movimiento, se debe verificar si es posible realizarlo siguiendo las reglas del entorno (límites del tablero, empuje de cajas y colisiones).

Implemente la función isValidMove, que reciba el estado actual y el movimiento a realizar, devolviendo True si es válido o False en caso contrario.

isValidMove :: State -> Move -> Bool

Casos de prueba:

Considerando el estado base:

st = ((2,2), (2,3), [(2,4)])

(Robot en (2,2), Caja Objetivo en (2,3) y una Caja de Bloqueo justo detrás en (2,4))

1) Movimiento simple a una casilla vacía (Arriba).

isValidMove st U

Retorno: True (El Robot pasaría de (2,2) a (1,2) sin problema).

2) Empuje inválido (Doble Caja): Intenta empujar a la derecha.

isValidMove st R

Retorno: False (El Robot intenta empujar la Caja Objetivo, pero choca con la Caja de Bloqueo en 2,4).

3) Empuje fuera del tablero:

Asumiendo un estado al borde inferior: $st_borde = ((4,4), (5,4), [])$

isValidMove ((4,4), (5,4), []) D

Retorno: False (Al bajar, empujaría la Caja Objetivo a la fila 6, lo cual está fuera del tablero).

Parte 3 - Ejecución de Movimiento

Implemente la lógica para alterar el estado del almacén. Esta función toma el estado actual y aplica un movimiento, devolviendo el nuevo estado. **Asuma que el movimiento ya fue validado por isValidMove.** Recuerde que si el Robot se mueve hacia una caja, la coordenada de esa caja también debe actualizarse.

applyMove :: State -> Move -> State

Casos de prueba:

Considerando el estado base: $st = ((2,2), (2,3), [(4,4)])$

1) Movimiento simple (Mover abajo)

applyMove st D

Retorno: $((3,2), (2,3), [(4,4)])$

2) Empujar Caja Objetivo (Mover derecha)

applyMove st R

Retorno: $((2,3), (2,4), [(4,4)])$

(El Robot toma el lugar de la caja, y la caja se desplaza una casilla a la derecha).

3) Empujar Caja de Bloqueo

Asumiendo el estado: $st_bloqueo = ((4,3), (1,1), [(4,4)])$

applyMove ((4,3), (1,1), [(4,4)]) R

Retorno: $((4,4), (1,1), [(4,5)])$

(El Robot empuja la Caja de Bloqueo hacia la derecha).

Parte 4 - Mejor Solución

Implementar la función `solveWarehouse` para encontrar la secuencia de estados óptima que permite llevar la Caja Objetivo a la coordenada (5,5).

La función recibe el estado inicial. Debe devolver una tupla donde la primera posición indica la cantidad total de movimientos realizados, y la segunda es una lista con la secuencia de estados (el camino) desde el estado inicial hasta el estado ganador. *Debe utilizar un algoritmo de búsqueda en anchura (BFS) para garantizar la solución más corta.* Si no hay solución, devuelva (0, []).

`solveWarehouse :: State -> (Int, [State])`

Casos de prueba:

1) Caso simple (1 movimiento para ganar):

El Robot empuja a la derecha y la Caja Objetivo llega a (5,5).

`solveWarehouse ((5,3), (5,4), [])`

Retorno: (1, [((5,3), (5,4), []), ((5,4), (5,5), [])])

2) Caso de posicionamiento simple (3 movimientos):

El Robot debe rodear la caja para ubicarse detrás de ella y empujarla hacia la meta (subir, mover a la derecha y empujar hacia abajo).

`solveWarehouse ((4,4), (4,5), [])`

Retorno: (3, [((4,4), (4,5), []), ((3,4), (4,5), []), ((3,5), (4,5), []), ((4,5), (5,5), [])])

Nota: El Robot no puede empujar la caja directamente hacia la meta desde su posición inicial. Debe subir a (3,4), moverse a la derecha a (3,5) para ubicarse justo arriba de la caja, y finalmente empujarla hacia abajo para que alcance la coordenada (5,5).

3) Caso sin solución (Atrapado):

El Robot y la Caja Objetivo están encerrados en la esquina por Cajas de Bloqueo.

`solveWarehouse ((0,0), (0,1), [(0,2), (1,1), (1,2)])`

Retorno: (0, [])

Puntuación:

- Parte 1, 1 pto.
- Parte 2 y 3, 2 ptos c/u.
- Parte 4, 14 ptos.
- Documentación o Readme, 1 pto.

Consideraciones para la entrega

- Reutilizar las funciones *isValidMove* y *applyMove*.
- Investigar y documentar el uso de la expresión *deriving* (*Show*, *Eq*).
- Gestionar los estados visitados usando listas.
- Realice las validaciones adicionales que considere necesarias.
- Sólo se pueden utilizar operaciones del Prelude de Haskell. No se permite *Data.Map* ni *Data.Set*. Deberá gestionar los estados visitados usando listas.
- Las funciones solicitadas deben estar definidas dentro de un único archivo llamado **Proyecto1.hs** y las funciones deben tener el mismo nombre y número de argumentos descritos en el enunciado.
- Las respuestas de las funciones deben ser iguales a las solicitadas en el enunciado, de no ser así el script de pruebas arrojará error y se calificará como incorrecto.
- Utilice comentarios en Haskell para separar las soluciones de las preguntas y documentar lo que considere necesario. En Haskell hay dos tipos de comentarios: línea (*--*) y bloque (*{ - y - }*). Incluya un comentario de encabezado con sus datos en el archivo.
- No está permitido utilizar soluciones de terceros (incluyendo soluciones generadas por LLMs). Cualquier material consultado o prompts para ideas de solución o documentación debe ser referenciado, indicando para qué sirvió. Esto lo pueden documentar en un archivo en formato *markdown* (**README.md**) o directamente como comentarios en **Proyecto1.hs**. No están permitidos documentos en ningún otro formato.
- El proyecto se puede realizar en grupos de **hasta dos estudiantes de cualquier sección** y la fecha de entrega será el **lunes 01/06/2025**, hasta las 11:59 pm (VET).
- El proyecto se debe adjuntar por e-mail a la dirección `ldpucv@gmail.com` usando como asunto: `[Proyecto Haskell] <Apellido(s)> <Nombre(s)>, <Cédula1> - <Apellido(s)> <Nombre(s)>, <Cédula2>`.

José Yvimas, Mayo 2026